Temperature and Humidity Monitoring and Alert System

A PROJECT REPORT

Soumy Mittal (DL.AI.U4AID25095)

in partial fulfilment for the award of ceridts for subjects

Introduction to Electronics(23ECE113) and
Introduction to Internet Of Things(23AID212)

BACHELOR OF TECHNOLOGY (B.Tech) IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AIDS)

Under the guidance of

Dr. Abhishek Kumar

Submitted to

AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE

FARIDABAD – 121002, June 2026

Abstract

Environmental monitoring plays a crucial role in ensuring safety, reliability, and operational efficiency across industrial, agricultural, laboratory, and residential environments. Continuous observation of temperature and humidity parameters enables early detection of abnormal conditions that may lead to equipment failure, product degradation, or unsafe operating environments.

This project presents the design and implementation of an **IoT-Based Temperature and Humidity Monitoring and Alert System** using an **ESP32 microcontroller** and a **DHT11 temperature-humidity sensor**. The proposed system continuously measures environmental conditions and compares the acquired readings against predefined threshold values. Whenever the sensed temperature or humidity exceeds the permissible operating range, the system generates immediate local alerts through an LED indicator and a piezoelectric buzzer. In addition, remote email notifications are transmitted using Gmail SMTP services over a Wi-Fi network, enabling users to monitor environmental conditions from remote locations.

The firmware is developed using the Arduino framework and incorporates automatic Wi-Fi reconnection, threshold evaluation, and timer-based email notification mechanisms. To validate the system design, comparator-based threshold detection was simulated using LTspice, while overall hardware connectivity and functionality were verified through Tinkercad simulation. Experimental results demonstrated reliable sensor monitoring, accurate threshold detection, timely alert generation, and successful remote notification delivery.

The developed system offers a low-cost, scalable, and reliable solution for real-time environmental monitoring. Furthermore, the architecture provides a foundation for future enhancements such as cloud-based dashboards, data logging, machine learning-based prediction, and advanced IoT analytics.

Keywords: Internet of Things (IoT), ESP32, DHT11 Sensor, Temperature Monitoring, Humidity Monitoring, Environmental Monitoring, Email Alert System, Wireless Sensor Networks, LTspice, Tinkercad.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The rapid growth of the Internet of Things (IoT) has enabled the development of intelligent monitoring systems capable of collecting, processing, and transmitting environmental data in real time. Temperature and humidity are among the most important environmental parameters, influencing the performance of industrial equipment, agricultural processes, storage facilities, healthcare environments, and residential spaces. Continuous monitoring of these parameters is therefore essential for ensuring operational safety, maintaining product quality, and preventing adverse environmental conditions.

This project presents the design and implementation of an **IoT-Based Temperature and Humidity Monitoring and Alert System** utilizing an **ESP32 microcontroller** and a **DHT11 temperature-humidity sensor**. The system continuously acquires environmental data and compares the measured values against predefined threshold limits. Whenever the sensed parameters exceed the permissible range, the system generates both local and remote alerts, thereby enabling timely intervention and reducing the risk of equipment failure, material degradation, or unsafe operating conditions.

The proposed system incorporates a multi-layer alert mechanism consisting of:

- **Local Physical Alerts:** A visual indication through an LED and an audible warning through a piezoelectric buzzer, activated immediately when threshold violations are detected.
- **Remote Notifications:** Automated email alerts transmitted through the ESP32 using Gmail SMTP services with SSL/TLS security. Notifications are generated periodically while the abnormal condition persists, ensuring that the user remains informed even when away from the monitoring location.

The ESP32 firmware is designed using a non-blocking architecture to ensure continuous sensor monitoring, responsive alert generation, and reliable network communication. Automatic Wi-Fi reconnection mechanisms further enhance system robustness by enabling recovery from temporary network disruptions without requiring manual intervention.

In addition to hardware implementation, the project includes simulation and verification using industry-standard tools. The threshold detection mechanism is analyzed through LTspice-based comparator circuit simulations, while overall hardware connectivity and operational logic are validated using the Tinkercad simulation environment. Furthermore, environmental data collected through the system can be stored and utilized for future analysis and predictive modeling, demonstrating the potential integration of IoT monitoring with data-driven decision-making techniques.

1.1 Objectives

The primary objectives of this project are:

1. To measure and monitor ambient temperature and humidity in real time using the DHT11 sensor.
2. To generate immediate local alerts through an LED and buzzer whenever environmental parameters exceed predefined threshold values.
3. To provide remote monitoring capabilities through automated email notifications using the ESP32 Mail Client library.
4. To implement a reliable alert mechanism that continues issuing notifications at regular intervals until normal environmental conditions are restored.
5. To design and validate a comparator-based threshold detection circuit using LTspice simulation.
6. To prototype and verify the complete hardware system using the Tinkercad simulation platform.
7. To demonstrate the application of IoT technologies in environmental monitoring and intelligent alert systems.

Chapter 2

System Architecture and Design

This chapter presents the overall architecture and operational workflow of the proposed IoT-based Temperature and Humidity Monitoring System. The system is designed to continuously monitor environmental parameters, generate local alerts when abnormal conditions are detected, and communicate critical information to users through remote email notifications. The architecture integrates sensing, processing, alert generation, and wireless communication modules into a unified monitoring platform.

2.1 System Architecture

The proposed system follows a layered architecture that separates sensing, processing, alert generation, and communication functionalities. Such a modular design improves system reliability, simplifies troubleshooting, and facilitates future upgrades.

2.1.1 Block Diagram Overview

The complete system architecture is illustrated in Figure 2.1. The architecture consists of three primary functional layers:

1. **Sensing Layer:** The DHT11 sensor continuously measures ambient temperature and relative humidity and transmits the acquired data to the ESP32 microcontroller.
2. **Processing and Alert Layer:** The ESP32 processes the sensor readings, compares them against predefined threshold values, and activates the comparator-transistor circuit responsible for driving the LED and buzzer alert mechanisms.
3. **Communication Layer:** The integrated Wi-Fi module of the ESP32 establishes connectivity with the Gmail SMTP server, enabling remote email notifications whenever abnormal environmental conditions are detected.

The interaction among these layers ensures real-time environmental monitoring, rapid

local response, and reliable remote notification.

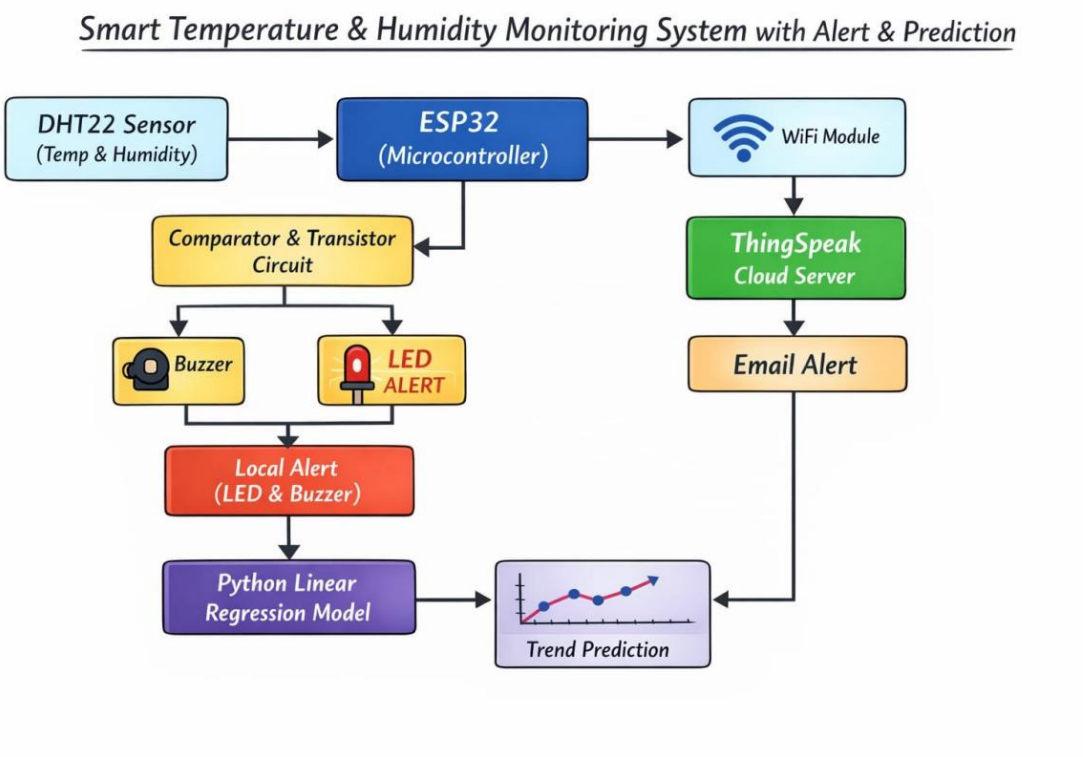

Figure 2.1: System block diagram showing data flow from the DHT11 sensor through the ESP32 microcontroller to local alert devices and cloud-based notification services.

2.1.2 Functional Analysis of the Architecture

The block diagram highlights the major data paths and control signals within the system. The following observations can be made:

- Environmental data is acquired through the DHT11 sensor and transmitted directly to the ESP32 for processing.
- The ESP32 serves as the central controller, performing threshold evaluation and decision-making operations.
- Upon detection of abnormal conditions, the controller activates both visual and audible warning devices through the comparator-transistor driver circuit.
- Simultaneously, the Wi-Fi subsystem establishes communication with the Gmail SMTP server to deliver remote email alerts.
- The architecture can be extended to incorporate predictive analytics, where historical sensor data is processed using machine learning models to forecast future environmental trends and potential threshold violations.

2.2 Firmware Design and Operational Flow

The firmware running on the ESP32 is responsible for sensor acquisition, threshold evaluation, alert generation, and network communication. The program is implemented using a non-blocking execution strategy to ensure continuous monitoring while maintaining responsiveness to environmental changes.

2.2.1 Logic Flowchart

The firmware execution sequence is illustrated in Figure 2.2. The flowchart describes the complete decision-making process implemented within the ESP32.

The operational sequence is summarized as follows:

1. Verify Wi-Fi connectivity and automatically reconnect if the network connection is unavailable.
2. Acquire temperature and humidity measurements from the DHT11 sensor.
3. Validate the acquired sensor readings to detect communication errors or invalid values.
4. Compare measured values against predefined upper and lower threshold limits.
5. Activate or deactivate the LED and buzzer according to the threshold evaluation results.
6. Generate and transmit email notifications when abnormal environmental conditions persist.
7. Repeat the monitoring cycle after a predefined sampling interval.

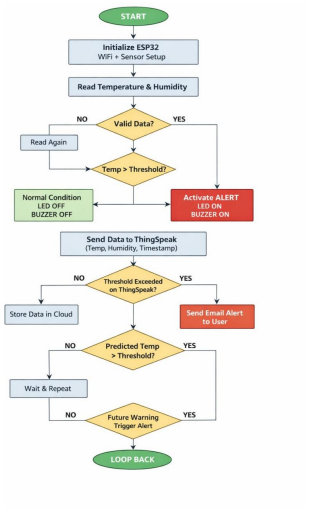

Figure 2.2: Firmware logic flowchart illustrating the decision-making process implemented within the ESP32 monitoring loop.

2.2.2 Operational Workflow

The firmware continuously executes a monitoring loop in which sensor data is sampled, analyzed, and acted upon in real time. When the measured temperature or humidity exceeds the configured threshold values, the ESP32 immediately triggers local alerts through the LED and buzzer. In addition, email notifications are transmitted periodically to ensure that the user remains informed until environmental conditions return to the acceptable operating range.

The implementation of automatic Wi-Fi reconnection and periodic alert generation enhances the robustness and reliability of the system, making it suitable for long-term deployment in industrial, agricultural, and residential monitoring applications.

Chapter 3

Hardware Design and Implementation

3.1 Hardware Components

The hardware architecture consists of a sensing unit, a processing unit, alert-generation devices, and supporting circuitry. Together, these components enable real-time environmental monitoring and notification capabilities.

3.1.1 Bill of Materials

Table 3.1: Bill of Materials

#	Component	Description / Specification	Qty	Unit Cost (Rs.)	Total (Rs.)
1	**ESP32 Dev Board**	Dual-core Xtensa LX6, 240 MHz, integrated Wi-Fi and Bluetooth	1	450	450
2	**DHT11 Sensor**	Temperature: 0–50 °C (±2 °C), Humidity: 20–90 % RH (±5 %)	1	120	120
3	**LED (Red)**	5 mm indicator LED, forward voltage approximately 2 V	1	5	5
4	**Piezo Buzzer**	Active buzzer operating at 5 V with audible alert output	1	30	30
5	**Resistor (220 Ω)**	Current-limiting resistor for LED protection	1	2	2
6	**Breadboard + Wires**	Standard prototyping breadboard and jumper wires	1	150	150
				Total Project Cost	**Rs.757**

Table 3.1 presents the complete list of hardware components used in the implementation of the proposed system.

3.1.2 Component Description

The major hardware components and their roles within the system are summarized below:

- **ESP32 Development Board:** Acts as the central processing unit of the system. It acquires sensor data, evaluates threshold conditions, controls alert devices, and manages Wi-Fi communication for email notifications.
- **DHT11 Sensor:** Measures ambient temperature and relative humidity and provides digital output directly to the ESP32.
- **LED Indicator:** Provides visual feedback whenever an abnormal environmental condition is detected.
- **Piezo Buzzer:** Generates an audible warning signal to attract immediate user attention during alert conditions.
- **Resistor:** Limits current through the LED and protects it from excessive current flow.
- **Breadboard and Connecting Wires:** Facilitate rapid prototyping and circuit assembly without soldering.

3.2 Pin Configuration and Interfacing

3.2.1 ESP32 GPIO Pin Mapping

Table 3.2 summarizes the pin assignments used in the project.

Table 3.2: ESP32 GPIO Pin Mapping

Signal	GPIO Pin	Description
DHT11 Data	GPIO 4	Digital data line from DHT11 sensor
LED Control	GPIO 2	Drives the visual alert indicator through a 220 Ω resistor
Buzzer Control	GPIO 5	Controls the active piezo buzzer; HIGH logic level activates the buzzer
Power Supply	3V3 / VIN	Provides operating voltage to connected peripherals
Ground	GND	Common reference connection for all circuit components

Proper GPIO allocation is essential for reliable operation of the monitoring system. The

ESP32 communicates with the DHT11 sensor and controls the alert devices through dedicated GPIO pins.

3.2.2 Interfacing Considerations

The DHT11 sensor communicates with the ESP32 through a single-wire digital interface. The LED is connected through a current-limiting resistor to prevent damage caused by excessive current flow. The active buzzer is controlled directly through a GPIO pin and is activated whenever a threshold violation is detected.

A common ground connection is maintained among all components to ensure signal integrity and reliable operation. The selected GPIO assignments avoid boot-mode conflicts and support stable execution of the ESP32 firmware.

3.3 Component Selection and Comparative Analysis

The components used in the proposed system were selected after considering factors such as cost, availability, ease of integration, power consumption, accuracy, and suitability for IoT applications. Table 3.3 summarizes the rationale behind the selected components.

Table 3.3: Comparative Analysis of Selected Components

Selected Component	Alternative Component(s)	Reason for Selection
ESP32 Dev Board	Arduino UNO, NodeMCU ESP8266	ESP32 was selected because it provides integrated Wi-Fi and Bluetooth connectivity, higher processing speed (240 MHz dual-core processor), larger memory resources, and better suitability for IoT applications. Using ESP32 eliminates the need for an external Wi-Fi module, reducing overall system complexity and cost.
DHT11 Sensor	DHT22, BME280, LM35	DHT11 was chosen due to its low cost, widespread availability, ease of interfacing, and sufficient accuracy for educational and prototype-level temperature and humidity monitoring applications.
Red LED	Green LED, Blue LED, RGB LED	Red LEDs are commonly associated with warning and fault indications, offer high visibility, require a lower forward voltage (approximately 2 V), and consume less power compared to many other LED colors.

Selected Component	Alternative Component(s)	Reason for Selection
Piezo Buzzer	Passive Buzzer, Speaker Module	An active piezo buzzer can generate sound directly from a digital GPIO signal without requiring additional signal-generation circuitry, making implementation simple and reliable.
220 Ω Resistor	330 Ω, 470 Ω	The selected resistance value provides adequate current limiting while maintaining sufficient LED brightness and protecting both the LED and the ESP32 GPIO pins.
Breadboard	PCB Prototype Board	Breadboards allow rapid prototyping, circuit modification, and debugging without soldering, reducing development time and implementation cost.
Gmail SMTP Service	ThingSpeak Alerts, Telegram Bot, IFTTT Notifications	Gmail SMTP provides reliable email delivery, secure SSL/TLS communication, and direct notification capabilities without requiring additional cloud infrastructure.
Wi-Fi Communication	Bluetooth, ZigBee	Wi-Fi enables direct internet connectivity and supports remote email notifications from virtually any location with network access.

3.4 Hardware Prototype

3.4.1 Experimental Setup

The complete hardware prototype of the proposed IoT-based Temperature and Humidity Monitoring System is shown in Figure 3.1. The system consists of an ESP32 development board interfaced with a DHT11 temperature and humidity sensor, a red LED indicator, and an active piezoelectric buzzer. The circuit was assembled on a breadboard using jumper-wire connections to facilitate testing and debugging.

The ESP32 continuously acquires environmental data from the DHT11 sensor, processes the measurements, and activates the alert devices whenever the configured threshold values are exceeded. Simultaneously, the ESP32 connects to a Wi-Fi network and transmits email notifications to the user through Gmail SMTP services.

The experimental setup was used to verify sensor operation, threshold detection, local alert generation, and remote email notification functionality under various environmental conditions.

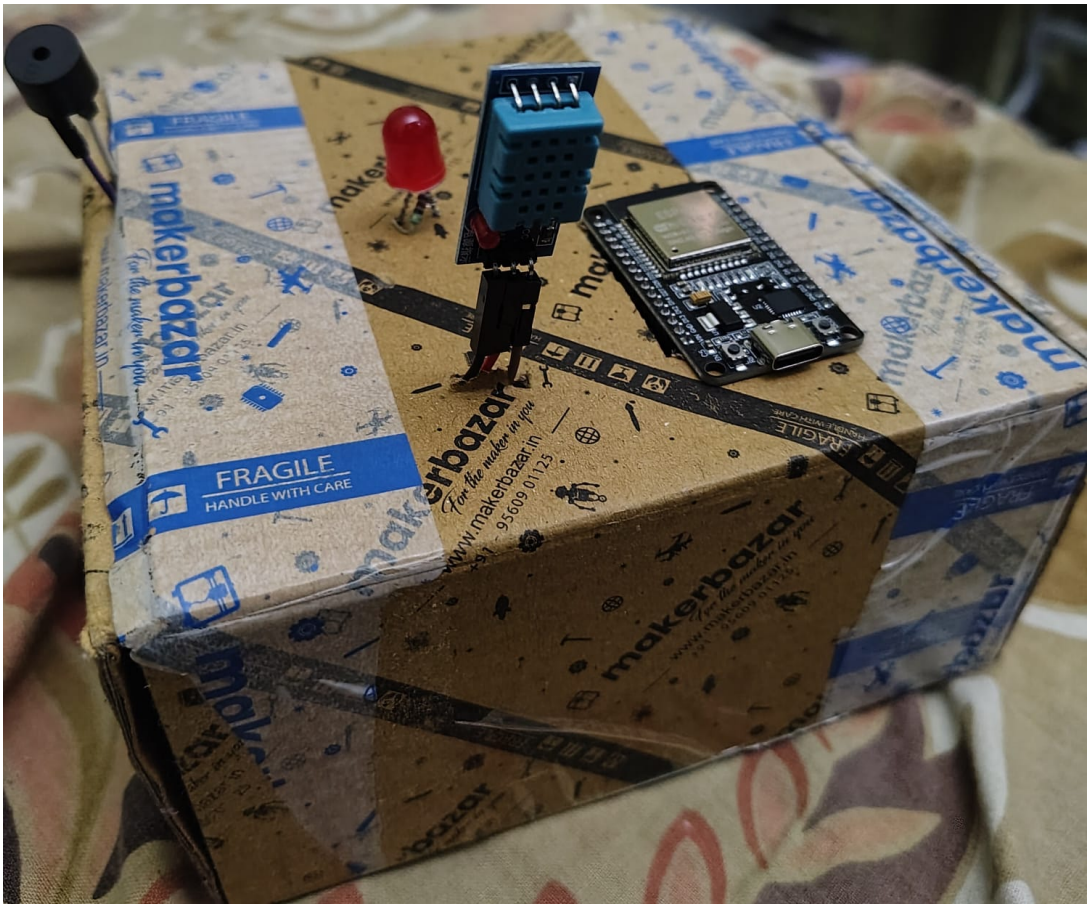

Figure 3.1: Hardware prototype of the IoT-based Temperature and Humidity Monitoring System.

Chapter 4

Circuit Design, Simulation and Validation

This chapter presents the design, simulation, and validation of the alert generation circuitry used in the proposed IoT-based Temperature and Humidity Monitoring System. Although threshold detection is primarily implemented in software using the ESP32 microcontroller, an equivalent hardware-based solution was developed and analyzed using LTspice. The objective of this study is to verify the threshold-switching behavior, investigate comparator operation, and validate the functionality of the overall alert mechanism. In addition, Tinkercad simulation was employed to verify hardware interconnections and system operation in a virtual prototyping environment.

4.1 LTspice-Based Comparator Circuit Design

A comparator circuit was designed and simulated using LTspice to model the threshold detection mechanism employed by the monitoring system. The comparator provides a hardware-level implementation of the decision-making process, allowing sensor signals to be converted into digital switching outputs capable of driving alert devices such as LEDs and buzzers.

4.1.1 Circuit Topology

Figure 4.1 illustrates the LTspice schematic of the comparator circuit.

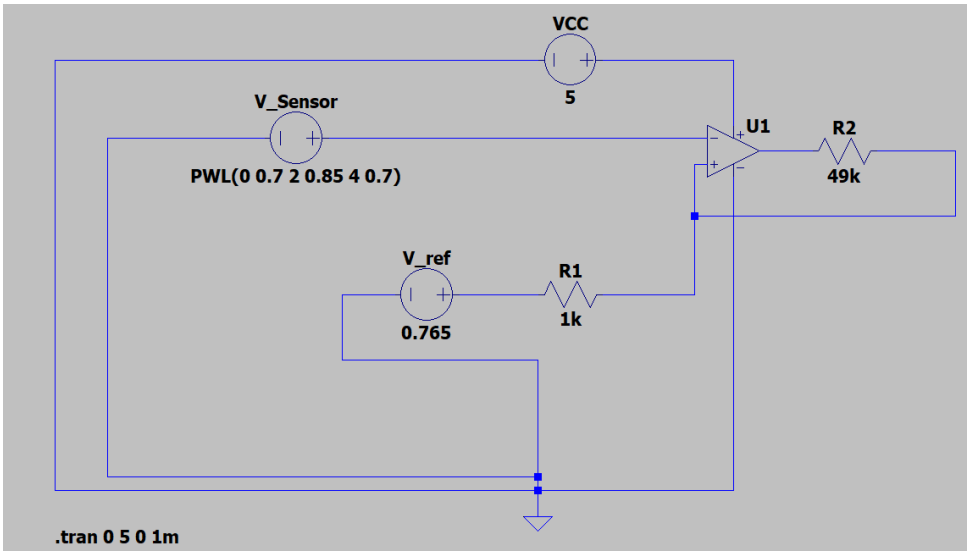

Figure 4.1: LTspice schematic of the comparator circuit used for threshold-based alert generation.

The circuit consists of the following major elements:

- **Sensor Voltage Source (V_{Sensor}):** A piecewise-linear (PWL) voltage source used to emulate variations in the sensor output over time.
- **Reference Voltage (V_{ref}):** A fixed threshold voltage of 0.765 V against which the sensor signal is continuously compared.
- **Comparator (U1):** An operational amplifier configured in open-loop mode to function as a voltage comparator.
- **Feedback Network (R1 and R2):** Provides positive feedback to introduce hysteresis and eliminate switching instability near the threshold level.
- **Power Supply:** A regulated 5 V supply used to power the comparator circuit.

4.1.2 Operating Principle

The comparator continuously compares the sensor voltage with the reference voltage. The output state is determined according to the following relationship:

$$V_{\text{out}} = \begin{cases} V_{CC}, & V_{\text{sensor}} > V_{\text{ref}} \\ 0, & V_{\text{sensor}} \leq V_{\text{ref}} \end{cases} \tag{4.1}$$

When the sensor voltage exceeds the reference voltage, the comparator output saturates toward the positive supply rail, indicating an alert condition. Conversely, when the sensor voltage falls below the threshold, the output returns to a logic-low state.

To improve switching stability, positive feedback is introduced through resistor R2, creat-

ing a hysteresis band that prevents rapid oscillation caused by noise or small fluctuations around the threshold voltage.

The hysteresis voltage is given by:

$$V_{hys} = \frac{V_{CC} R_1}{R_1 + R_2}$$

For the selected component values:

$$V_{\text{hys}} = \frac{5 \times 1000}{1000 + 49000} \approx 0.1\,V \tag{4.2}$$

This hysteresis improves noise immunity and ensures reliable comparator operation.

4.2 Simulation Results and Analysis

Transient analysis was performed to observe the dynamic response of the comparator circuit under varying sensor input conditions.

4.2.1 LTspice Waveform Analysis

Figure 4.2 presents the transient simulation results obtained from LTspice.

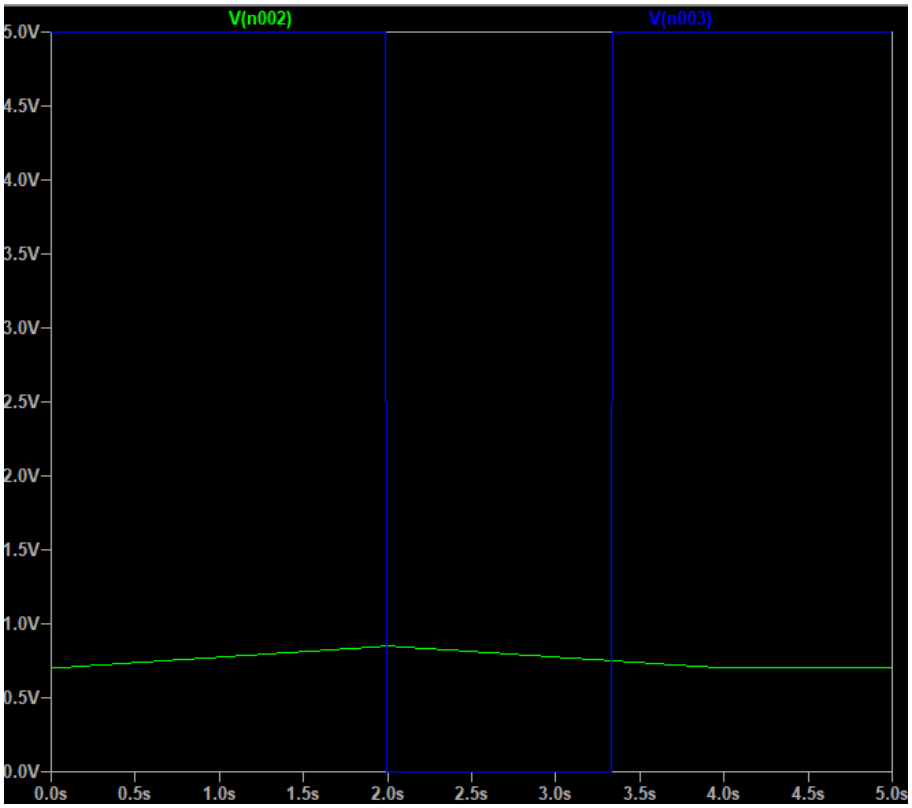

Figure 4.2: Transient simulation results showing sensor voltage and comparator output response.

The waveform demonstrates the successful conversion of an analog sensor signal into a digital switching output. The green trace represents the simulated sensor voltage, while the blue trace corresponds to the comparator output.

The following observations can be made:

- The sensor voltage gradually rises from approximately 0.70 V to 0.85 V before returning to its initial value.
- When the sensor voltage exceeds the threshold voltage of 0.765 V, the comparator output transitions rapidly to a high state.
- The output remains high while the sensor voltage stays above the threshold level.
- As the sensor voltage decreases below the threshold, the comparator output returns to the low state.
- The sharp transitions confirm proper comparator operation and demonstrate the effectiveness of the hysteresis network in preventing unwanted switching.

4.2.2 Discussion of Results

The simulation results validate the threshold-detection mechanism used in the proposed monitoring system. The comparator successfully distinguishes between normal and abnormal operating conditions and produces a clean digital output suitable for driving alert devices.

The inclusion of hysteresis significantly improves circuit stability by reducing sensitivity to noise and preventing oscillations around the threshold point. These characteristics make the circuit suitable for real-world environmental monitoring applications.

4.3 Tinkercad-Based Hardware Validation

To further verify the system design, the complete monitoring circuit was implemented and tested using the Tinkercad simulation environment.

4.3.1 Circuit Simulation Setup

Figure 4.3 illustrates the virtual prototype developed in Tinkercad.

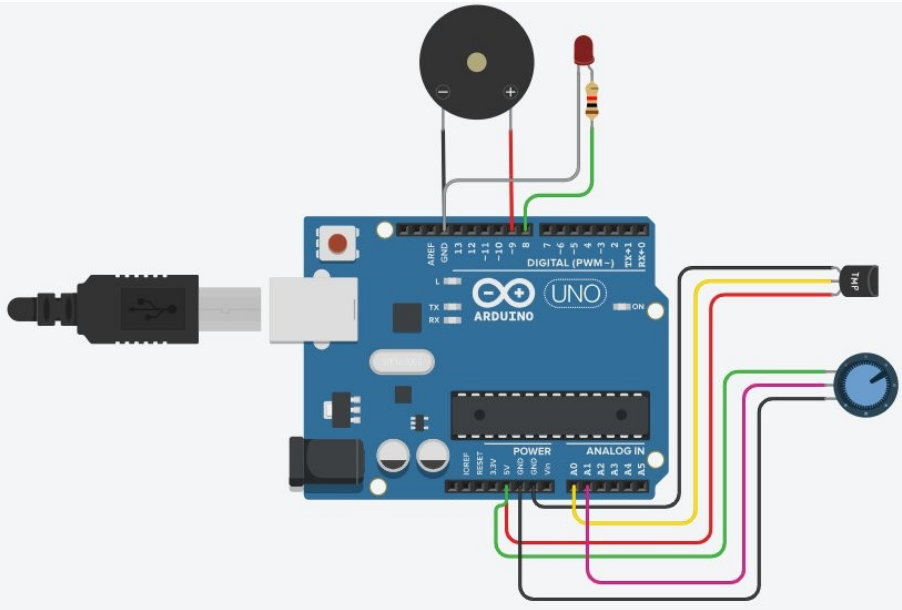

Figure 4.3: Tinkercad simulation of the temperature and humidity monitoring system.

The simulation includes:

- Arduino UNO microcontroller used as a functional substitute for the ESP32 during virtual prototyping.
- DHT11 temperature and humidity sensor.
- LED-based visual alert indicator.
- Piezoelectric buzzer for audible alerts.
- Potentiometer for emulating threshold adjustment.

4.3.2 Validation Results

The Tinkercad simulation confirmed the correctness of the hardware design and interconnections. The following observations were obtained:

- The DHT11 sensor communicated successfully with the controller.
- The LED activated correctly when threshold conditions were exceeded.
- The buzzer generated audible alerts during abnormal operating conditions.
- The potentiometer effectively simulated adjustable threshold values.
- The overall circuit behavior matched the expected operational characteristics of the final hardware implementation.

The successful LTspice and Tinkercad simulations provide confidence in both the analog threshold-detection circuitry and the complete monitoring system prior to physical deployment.

Chapter 5

Firmware Design and Software Implementation

This chapter describes the software architecture, firmware implementation, and communication mechanisms employed in the proposed IoT-based Temperature and Humidity Monitoring System. The firmware is responsible for acquiring sensor data, evaluating environmental conditions, generating local alerts, and transmitting remote email notifications. The software has been designed to operate reliably in real-time while maintaining continuous network connectivity and efficient resource utilization on the ESP32 platform.

5.1 Software Architecture

The firmware is developed using the Arduino framework in C/C++ and is deployed on the ESP32 microcontroller. The software architecture follows a modular approach in which sensing, decision-making, alert generation, and communication functions are organized into independent functional blocks. This design simplifies maintenance, improves readability, and facilitates future enhancements.

5.1.1 Software Libraries

The implementation relies on several external libraries that provide high-level interfaces for Wi-Fi communication, sensor acquisition, and SMTP email transmission. Table 5.1 summarizes the libraries used in the project.

Table 5.1: Firmware Libraries

Library	Purpose
`WiFi.h`	Provides wireless network connectivity and automatic reconnection support.
`DHT.h`	Interfaces with the DHT11 sensor and retrieves temperature and humidity measurements.
`ESP_Mail_Client.h`	Implements SMTP communication with SSL/TLS encryption for secure email delivery.

5.1.2 Firmware Workflow

The firmware operates in a continuous monitoring loop. Sensor readings are acquired at regular intervals and compared against predefined threshold values. When abnormal environmental conditions are detected, local and remote alert mechanisms are activated. The overall workflow includes:

1. Establish Wi-Fi connectivity.
2. Acquire temperature and humidity readings.
3. Validate sensor data.
4. Compare measurements with threshold limits.
5. Activate alert devices when required.
6. Generate and transmit email notifications.
7. Repeat the monitoring cycle.

5.2 Threshold Configuration

Threshold values define the acceptable operating range of environmental conditions. These values are implemented as configurable floating-point constants, enabling easy modification without altering the core application logic.

5.2.1 Temperature and Humidity Limits

The firmware defines upper and lower limits for both temperature and humidity.

```
1 float tempHigh = 30.0;
2 float tempLow  = 18.0;
3
4 float humHigh  = 50.0;
5 float humLow   = 30.0;
```

Listing 5.1: Threshold configuration constants

5.2.2 Danger Condition Evaluation

A danger condition is declared whenever at least one environmental parameter falls outside the specified operating range.

```
bool dangerCondition =
(temp > tempHigh || temp < tempLow ||
 hum  > humHigh  || hum  < humLow);
```

Listing 5.2: Danger condition evaluation

This logic enables the system to detect both excessively high and excessively low environmental conditions.

5.3 Alert Generation Mechanism

The alert subsystem is responsible for notifying users whenever environmental conditions exceed predefined thresholds. The system combines both local and remote notification methods to ensure reliable warning delivery.

5.3.1 Local Alert System

When an abnormal condition is detected, the ESP32 immediately activates:

- A red LED for visual indication.
- A piezoelectric buzzer for audible warning.

The alert devices remain active until all measured parameters return to their safe operating range.

5.3.2 Email Alert Logic

To prevent excessive email generation, the firmware incorporates a non-blocking timing mechanism based on the `millis()` function.

```
const unsigned long emailInterval = 60000;
unsigned long lastEmailTime = 0;
```

Listing 5.3: Email rate-limiting logic

Email notifications are transmitted at 60-second intervals while the danger condition persists. This approach balances user awareness with network and server resource utilization.

5.4 SMTP Communication Module

Remote notification is implemented using Gmail's SMTP service. The ESP32 establishes a secure SSL/TLS connection with the SMTP server and transmits formatted email messages containing current environmental conditions.

5.4.1 Email Transmission Process

The email transmission sequence consists of:

1. Creating an SMTP session.
2. Authenticating using Gmail credentials.
3. Constructing the email message.
4. Establishing an encrypted connection.
5. Sending the notification.
6. Closing the session.

5.4.2 SMTP Configuration

The Gmail SMTP server operates using SSL encryption on port 465. The firmware uses a Gmail App Password for authentication, ensuring compatibility with Google's security requirements.

5.5 Wi-Fi Connectivity Management

Reliable network communication is essential for remote monitoring functionality. The firmware continuously monitors Wi-Fi status and automatically attempts reconnection whenever connectivity is lost.

5.5.1 Automatic Reconnection Strategy

If the ESP32 detects a disconnected state, it immediately initiates a new connection attempt. This strategy improves system reliability and ensures that email notifications remain operational despite temporary network disruptions.

5.6 Complete Firmware Implementation

The complete firmware integrates sensor acquisition, threshold evaluation, alert generation, Wi-Fi management, and SMTP communication into a single embedded application. The source code is provided in Appendix A for reference and future development.

5.6.1 Program Structure

The firmware is organized around the following major functions:

- `setup()` – Hardware initialization and network configuration.
- `loop()` – Continuous monitoring and alert management.
- `sendEmail()` – SMTP-based notification service.

The modular implementation improves code readability, maintainability, and future scalability while ensuring reliable real-time monitoring performance.

Chapter 6

Results, Analysis and Performance Evaluation

6.1 System Operation

The developed monitoring system continuously acquires temperature and humidity measurements from the DHT11 sensor and compares the readings against predefined threshold values. Depending on the measured environmental conditions, the system operates either in a normal monitoring state or an alert state.

6.1.1 Normal Operating Condition

Under normal environmental conditions, all measured parameters remain within their predefined safe operating ranges:

- Temperature: $18\,^{\circ}\text{C} \leq T \leq 30\,^{\circ}\text{C}$
- Humidity: $30\% \leq H \leq 50\%$

During this mode of operation:

- The LED indicator remains OFF.
- The piezoelectric buzzer remains inactive.
- No email notifications are transmitted.
- Sensor readings are updated every two seconds.
- Environmental parameters are continuously displayed through the serial monitor for observation and debugging.

These observations confirm stable operation and low power consumption under normal environmental conditions.

6.1.2 Alert Condition

Whenever a measured parameter exceeds its permissible threshold, the system immediately enters the alert state.

The following actions are performed:

1. The LED indicator is switched ON.
2. The piezoelectric buzzer is activated.
3. An alert email is generated and transmitted.
4. Additional email notifications are sent every 60 seconds while the abnormal condition persists.

This mechanism ensures both local and remote awareness of environmental anomalies.

6.1.3 Email Notification Format

The email generated by the system contains the current environmental measurements, enabling rapid assessment of the detected condition.

```
Subject: Environment Alert

WARNING!

Temperature: 32.5 °C
Humidity: 55.0 %
```

6.1.4 System Recovery

Once environmental conditions return to the safe operating range, the ESP32 automatically deactivates all alert devices and returns to normal monitoring operation.

The recovery process includes:

- LED switched OFF.
- Buzzer switched OFF.
- Email notifications terminated.
- Continuous environmental monitoring resumed.

This autonomous recovery behavior eliminates the need for manual system reset or user intervention.

6.2 LTspice Simulation Results

The comparator circuit was analyzed using transient simulation in LTspice over a duration of five seconds with a simulation step size of 1 ms. The resulting waveforms verified the threshold detection and switching behavior of the proposed comparator-based alert circuit.

The simulation results demonstrated the following characteristics:

- **Fast Switching Response:** The comparator output transitioned within a single simulation time step, resulting in an effective switching latency of less than 1 ms.
- **Hysteresis Implementation:** The calculated hysteresis band of approximately 0.1 V prevented unwanted oscillations and noise-induced switching near the threshold voltage.
- **Digital Output Characteristics:** The comparator output exhibited clear rail-to-rail switching behavior, making it suitable for driving transistor-based alert circuitry.

The simulation confirms that the comparator circuit can reliably distinguish between normal and abnormal sensor conditions while maintaining stable operation.

6.3 Tinkercad Simulation Results

The complete monitoring system was further validated using the Tinkercad simulation platform. The virtual prototype successfully demonstrated correct hardware operation and component interaction.

The following observations were obtained:

- Successful acquisition of temperature and humidity data from the DHT11 sensor.
- Correct activation and deactivation of the LED and buzzer according to threshold conditions.
- Proper operation of the potentiometer used to emulate adjustable threshold levels.
- Reliable synchronization between sensor readings and alert outputs.

The simulation results verified the correctness of the hardware design prior to physical deployment.

6.4 Firmware Performance Analysis

The firmware implementation was evaluated with respect to responsiveness, communication reliability, and alert generation efficiency.

6.4.1 Threshold Evaluation

The ESP32 successfully evaluated all temperature and humidity readings against the configured threshold limits in real time. No missed threshold events were observed during testing.

6.4.2 Email Notification Performance

The non-blocking timer-based implementation ensured that:

- The first email notification was transmitted immediately after detection of an abnormal condition.
- Subsequent notifications were transmitted at fixed intervals of 60 seconds.
- Excessive email generation was prevented through rate limiting.
- Email delivery remained operational after temporary Wi-Fi reconnection events.

6.4.3 Wi-Fi Reliability

The automatic reconnection mechanism successfully restored network connectivity following temporary disconnections, thereby maintaining uninterrupted remote notification capability.

6.5 System Response Summary

Table 6.1 summarizes the response of the developed monitoring system under various operating conditions.

Table 6.1: System Response Summary

Condition	**LED**	**Buzzer**	**Email Notification**
Normal Operation	OFF	OFF	None
Temperature $> 30\,^{\circ}\mathrm{C}$	ON	ON	Sent every 60 s until normal
Temperature $< 18\,^{\circ}\mathrm{C}$	ON	ON	Sent every 60 s until normal
Humidity $> 50\,\%$	ON	ON	Sent every 60 s until normal
Humidity $< 30\,\%$	ON	ON	Sent every 60 s until normal
Sensor Read Error (NaN)	OFF	OFF	None; automatic retry after 2 s
Wi-Fi Disconnected	OFF	OFF	Automatic reconnection attempt initiated

6.6 Discussion

The experimental results demonstrate that the proposed monitoring system successfully fulfills all project objectives. The hardware subsystem, firmware implementation, and communication framework operated reliably throughout the testing process.

The LTspice simulation validated the comparator-based threshold detection mechanism, while the Tinkercad simulation confirmed proper hardware interfacing. Real-time monitoring, immediate alert generation, and periodic email notifications collectively provide a robust solution for environmental monitoring applications.

The successful integration of sensing, processing, alert generation, and remote communication demonstrates the practicality of IoT-based monitoring systems in industrial, agricultural, laboratory, and residential environments.

Chapter 7

Conclusion and Future Scope

This chapter summarizes the achievements of the proposed IoT-based Temperature and Humidity Monitoring System and discusses its limitations along with potential future enhancements. The developed system successfully integrates environmental sensing, local alert generation, wireless communication, and remote notification capabilities into a compact and cost-effective monitoring solution.

7.1 Conclusion

The primary objective of this project was to design and implement an IoT-based environmental monitoring system capable of continuously measuring temperature and humidity while providing timely alerts whenever abnormal conditions are detected. The developed system successfully achieved these objectives through the integration of the ESP32 microcontroller, DHT11 sensor, LED and buzzer alert mechanisms, and Gmail SMTP-based email notification services.

Experimental evaluation confirmed that the system is capable of:

- Monitoring temperature and humidity in real time.
- Detecting threshold violations accurately and reliably.
- Generating immediate local alerts through visual and audible indicators.
- Sending secure remote notifications using Wi-Fi connectivity and SMTP-based email communication.
- Automatically recovering from temporary network interruptions through Wi-Fi reconnection mechanisms.
- Operating continuously without manual intervention.

The LTspice simulation validated the threshold-detection circuitry and demonstrated stable comparator operation with hysteresis. The Tinkercad simulation verified hardware

connectivity and system functionality before physical implementation. Together, these simulation studies reduced development risk and improved design reliability.

The developed monitoring system therefore provides an effective, low-cost, and scalable solution for environmental monitoring applications in residential, industrial, agricultural, laboratory, and storage environments.

7.2 Advantages of the Proposed System

Several design choices contribute to the effectiveness and reliability of the proposed monitoring system.

7.2.1 Dual-Layer Alert Mechanism

The combination of local and remote notification methods enhances system dependability. Local alerts provide immediate awareness of abnormal conditions, while email notifications ensure that users can be informed even when they are physically away from the monitoring location.

7.2.2 Efficient Email Notification Strategy

The implementation of a 60-second notification interval prevents excessive email generation and avoids unnecessary SMTP server load while maintaining periodic status updates during prolonged alert conditions.

7.2.3 Responsive Firmware Architecture

The use of timer-based event handling allows the monitoring loop to remain responsive during system operation. Sensor acquisition and alert management can continue without significant interruption.

7.2.4 Automatic Network Recovery

The Wi-Fi reconnection mechanism enables the ESP32 to recover automatically from temporary network outages, improving system reliability and reducing the need for manual maintenance.

7.2.5 Simulation-Based Validation

The use of LTspice and Tinkercad simulations prior to hardware deployment provided valuable insight into circuit behavior and hardware integration, thereby reducing development time and troubleshooting effort.

7.3 Limitations of the Current System

Although the system performs effectively, several limitations were identified during development and testing.

Table 7.1: Current System Limitations

Limitation	Impact
DHT11 sensor accuracy limitations	Measurement precision is lower compared to industrial-grade sensors.
Hardcoded Wi-Fi and email credentials	Reduces security and flexibility during deployment.
Absence of local data storage	Historical measurements cannot be reviewed without external logging.
Single-recipient notification system	Alerts can only be delivered to one configured email address.
No local display interface	Users cannot directly view environmental parameters on the device.
No predictive analytics capability	The system reacts to threshold violations but cannot forecast future events.
No cloud dashboard integration	Real-time data visualization is limited.

7.4 Future Scope

Several improvements can be incorporated to enhance system functionality, accuracy, and scalability.

Table 7.2: Future Enhancements

Proposed Enhancement	Expected Benefit
Replace DHT11 with DHT22 or BME280	Improved measurement accuracy and wider sensing range.
Secure credential storage using NVS	Enhanced security and easier device configuration.
SD card or SPIFFS logging	Long-term storage of environmental data.
Multi-recipient email support	Simultaneous notification of multiple users.
Machine learning prediction models	Forecasting of future threshold violations and environmental trends.
Mobile application support	Instant push notifications and remote control capabilities.
MQTT-based communication	Scalable integration with industrial IoT platforms.

7.5 Final Remarks

The successful implementation of the proposed IoT-based Temperature and Humidity Monitoring System demonstrates the practical application of embedded systems, wireless communication, and cloud-enabled notification technologies for environmental monitoring. The project provides a strong foundation for future research and development in smart monitoring systems, predictive maintenance, and intelligent IoT-based automation solutions.

With the addition of advanced sensing technologies, cloud analytics, and machine learning techniques, the proposed system can evolve into a fully featured smart environmental monitoring platform suitable for large-scale industrial and commercial deployments.

References

[1] Espressif Systems, *ESP32 Technical Reference Manual*, Rev. 5.2, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[2] Aosong Electronics, *DHT11 Humidity & Temperature Sensor Product Manual*, 2010.

[3] Mobizt, *ESP-Mail-Client Arduino Library Documentation*, GitHub. [Online]. Available: https://github.com/mobizt/ESP-Mail-Client

[4] Texas Instruments, *LM393 Dual Differential Comparator Datasheet*, SNOSC04C, 2016. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm393.pdf

[5] Autodesk, *Tinkercad Circuits Online Simulator*. [Online]. Available: https://www.tinkercad.com

[6] Analog Devices, *LTspice Simulation Software Documentation*. [Online]. Available: https://www.analog.com/en/design-center/design-tools-and-calculators/ltspice-simulator.html

[7] Arduino, *Arduino Reference — millis()*. [Online]. Available: https://www.arduino.cc/reference/en/language/functions/time/millis/

[8] Google, *Sign in with App Passwords — Google Account Help*. [Online]. Available: https://support.google.com/accounts/answer/185833

.1 GPIO Wiring Quick-Reference

```
ESP32 Pin  <-->  Component
-----------      ---------
GPIO 4     <-->  DHT11 DATA  (+ 10k pull-up to 3V3)
GPIO 2     <-->  LED Anode   (+ 220R series to GND)
GPIO 5     <-->  Buzzer +    (Buzzer - to GND)
3V3        <-->  DHT11 VCC
GND        <-->  DHT11 GND, LED Cathode (via R), Buzzer GND
```

.2 Gmail App Password Setup

1. Enable 2-Step Verification on the Gmail account.
2. Navigate to **Google Account → Security → App passwords**.
3. Create a new app password for *Mail* on *Other (custom name)*.
4. Copy the 16-character password (spaces included as displayed).
5. Paste into the `AUTHOR_PASSWORD` macro in the firmware.

Appendix A

ESP32 Firmware Source Code

The complete source code used for implementing the IoT-Based Temperature and Humidity Monitoring and Alert System is presented below.

```

#include <WiFi.h>
#include <ESP_Mail_Client.h>
#include <DHT.h>

// ===== WIFI =====
#define WIFI_SSID "**********"
#define WIFI_PASSWORD "**********"

// ===== EMAIL =====
#define SMTP_HOST "smtp.gmail.com"
#define SMTP_PORT 465

#define AUTHOR_EMAIL "soumymittal09@gmail.com"
#define AUTHOR_PASSWORD "yoaz qiuo xsal whxd"

#define RECIPIENT_EMAIL "soumymittal09@gmail.com"

// ===== SENSOR =====
#define DHTPIN 4
#define DHTTYPE DHT11

// ===== OUTPUTS =====
#define LED_PIN 2
#define BUZZER_PIN 5

DHT dht(DHTPIN, DHTTYPE);
```

```
28
29 // ===== THRESHOLDS =====
30 float tempHigh = 30.0;
31 float tempLow = 18.0;
32
33 float humHigh = 50.0;
34 float humLow = 30.0;
35
36 // ===== EMAIL TIMER =====
37 unsigned long lastEmailTime = 0;
38
39 const unsigned long emailInterval = 60000; // 60 seconds
40
41 // ===== SMTP SESSION =====
42 SMTPSession smtp;
43
44 // ===== EMAIL FUNCTION =====
45 void sendEmail(String msg) {
46
47   SMTP_Message message;
48
49   message.sender.name = "ESP32 Alert System";
50   message.sender.email = AUTHOR_EMAIL;
51
52   message.subject = "Environment Alert";
53
54   message.addRecipient("User", RECIPIENT_EMAIL);
55
56   message.text.content = msg.c_str();
57
58   ESP_Mail_Session session;
59
60   session.server.host_name = SMTP_HOST;
61   session.server.port = SMTP_PORT;
62
63   session.login.email = AUTHOR_EMAIL;
64   session.login.password = AUTHOR_PASSWORD;
65
66   session.time.ntp_server = "pool.ntp.org,time.nist.gov";
67   session.time.gmt_offset = 5.5;
68   session.time.day_light_offset = 0;
69
70   Serial.println("Connecting to SMTP Server...");
```

```
71
72    if (!smtp.connect(&session)) {
73
74      Serial.println("SMTP Connection Failed!");
75      Serial.println(smtp.errorReason());
76
77      return;
78    }
79
80    if (!MailClient.sendMail(&smtp, &message)) {
81
82      Serial.println("Email Sending Failed!");
83      Serial.println(smtp.errorReason());
84    }
85
86    else {
87
88      Serial.println("Email Sent Successfully!");
89    }
90
91    smtp.closeSession();
92  }
93
94  // ===== SETUP =====
95  void setup() {
96
97    Serial.begin(115200);
98
99    pinMode(LED_PIN, OUTPUT);
100   pinMode(BUZZER_PIN, OUTPUT);
101
102   digitalWrite(LED_PIN, LOW);
103   digitalWrite(BUZZER_PIN, LOW);
104
105   dht.begin();
106
107   delay(2000);
108
109   Serial.println("DHT11 Initialized");
110
111   Serial.println("Connecting to WiFi");
112
113   WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
114
115   int attempts = 0;
116
117   while (WiFi.status() != WL_CONNECTED && attempts < 20) {
118
119     delay(1000);
120
121     Serial.print(".");
122
123     attempts++;
124   }
125
126   if (WiFi.status() == WL_CONNECTED) {
127
128     Serial.println("\nWiFi Connected!");
129
130     Serial.print("IP Address: ");
131     Serial.println(WiFi.localIP());
132   }
133
134   else {
135
136     Serial.println("\nWiFi Connection Failed!");
137   }
138 }
139
140 // ===== LOOP =====
141 void loop() {
142
143   if (WiFi.status() != WL_CONNECTED) {
144
145     Serial.println("WiFi Reconnecting...");
146
147     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
148
149     delay(5000);
150
151     return;
152   }
153
154   float temp = dht.readTemperature();
155   float hum = dht.readHumidity();
156
```

```
157   if (isnan(temp) || isnan(hum)) {
158
159     Serial.println("DHT SENSOR ERROR!");
160
161     delay(2000);
162
163     return;
164   }
165
166   Serial.println("\n====== SENSOR DATA ======");
167
168   Serial.print("Temperature: ");
169   Serial.print(temp);
170   Serial.println(" °C");
171
172   Serial.print("Humidity: ");
173   Serial.print(hum);
174   Serial.println(" %");
175
176   bool dangerCondition =
177       (temp > tempHigh || temp < tempLow ||
178        hum > humHigh || hum < humLow);
179
180   if (dangerCondition) {
181
182     digitalWrite(LED_PIN, HIGH);
183     digitalWrite(BUZZER_PIN, HIGH);
184
185     Serial.println("ALERT ON");
186
187     if (millis() - lastEmailTime >= emailInterval ||
188         lastEmailTime == 0) {
189
190       String msg = "WARNING!\n\n";
191
192       msg += "Temperature: ";
193       msg += String(temp);
194       msg += " °C\n";
195
196       msg += "Humidity: ";
197       msg += String(hum);
198       msg += " %";
199
```

```
200         sendEmail(msg);
201
202         lastEmailTime = millis();
203
204         Serial.println("EMAIL SENT");
205       }
206     }
207
208     else {
209
210       digitalWrite(LED_PIN, LOW);
211       digitalWrite(BUZZER_PIN, LOW);
212
213       Serial.println("ALERT OFF");
214     }
215
216     Serial.println("==========================");
217
218     delay(2000);
219 }
```

Listing A.1: ESP32 Temperature and Humidity Monitoring System